Hospital Capacity Reporting Portal for Arogya-SMC: Complete Design Document

1. Overview

The **Arogya-SMC Hospital Portal** is a lightweight web application that enables hospital administrators to submit daily capacity data (beds, ICU, ventilators, oxygen) and disease admission counts to the central system. This data powers the citizen app's facility lookup and the municipal dashboard's resource monitoring features.

Key Objectives:

- Provide a simple, fast interface for busy hospital staff
- Capture standardized data aligned with national reporting requirements
- Enable real-time visibility of hospital resources for SMC
- Maintain audit trails for accountability

---

2. Technology Stack (100% Free)

Core Framework

Component	Technology	Justification	Cost
Framework	Next.js 15+ (App Router)	React-based, server-side rendering, built-in API routes, excellent performance	Free
Language	TypeScript 5+	Type safety, better developer experience	Free
Styling	Tailwind CSS 4+	Utility-first, rapid UI development, responsive design	Free
UI Components	shadcn/ui	Accessible, customizable component library built on Radix UI	Free
Deployment	Vercel (Hobby)	Free tier with generous limits, seamless Next.js integration	Free

State Management & Data Fetching

Component	Technology	Purpose
Server State	TanStack Query (React Query) v5	Data fetching, caching, background updates
Form Management	React Hook Form + Zod	Performant forms with validation
HTTP Client	Axios	API requests with interceptors

Authentication & Security (Prototype)

Component	Technology	Purpose
Authentication	NextAuth.js (Auth.js)	Simple email/password for demo
JWT Handling	js-cookie + jose	Store and verify tokens

Development Tools

Tool	Purpose
ESLint	Code linting
Prettier	Code formatting
Husky	Git hooks
Postman	API testing

---

3. Understanding Hospital Capacity Reporting Requirements

Industry Standards & Best Practices

Based on analysis of hospital capacity reporting systems:

Data Reporting Cadence:

- Real-time updates every **15 minutes** for critical resources
- Daily snapshots for regulatory compliance
- Bed availability should reflect **23:59 snapshot** for consistency

Core Data Elements (from HL7 FHIR standards) :

- Total bed capacity by type (medical/surgical, ICU, pediatric, etc.)
- Available/occupied beds
- Ventilator availability
- Oxygen availability
- Staffed vs. unstaffed beds

Bed Status Definitions :

- **Available Bed Days**: Staffed beds that are operational (occupied or unoccupied)
- **Closed Bed Days**: Staffed beds temporarily closed and unoccupied
- **Permanently Closed Beds**: Removed from reporting entirely

What Existing Systems Track

Category	Data Points
Capacity	Bed occupancy, ICU availability, ventilator counts, oxygen status
Patient Flow	Admissions, discharges, delayed transfers

Category	Data Points
Clinical	Disease-specific admissions (malaria, dengue, COVID-like)
Operational	Staffing levels, equipment status, emergency alerts

4. Features & Functions

Core Features (Must-Have for March 23)

Feature	Description	Priority
F1: Secure Login	Hospital-specific login with email/password	HIGH
F2: Dashboard View	Overview of last submission, quick actions	HIGH
F3: Capacity Form	Daily bed/ICU/ventilator/oxygen reporting	HIGH
F4: Disease Counts	Report admissions by disease type	HIGH
F5: Submission History	View past submissions with edit capability	HIGH
F6: Last Submission Pre-fill	Auto-populate form with yesterday's values	HIGH
F7: Form Validation	Client + server validation for data integrity	HIGH
F8: Success/Failure Feedback	Clear submission confirmation or errors	HIGH

Secondary Features (Post-Prototype)

Feature	Description
Audit Log	Complete history with timestamps and user info
2FA	Two-factor authentication for security
Reporting	Download submission history as CSV
Multiple Users	Different roles within hospital (admin, nurse, etc.)
EMR Integration	Future API connection to hospital systems

5. Data Model

Based on FHIR standards for bed capacity reporting :

```
// Hospital Capacity Report
interface CapacityReport {
  reportId: string;
  hospitalId: string;
  hospitalName: string;
  reportDate: string; // ISO date YYYY-MM-DD
```

```
  submittedAt: string; // ISO datetime
  submittedBy: string; // user email/id

  // Bed Capacity
  beds: {
    total: number;      // Total staffed beds
    available: number;  // Currently available
    occupied?: number;  // Calculated: total - available
  };

  // ICU Capacity
  icu: {
    total: number;
    available: number;
  };

  // Ventilators
  ventilators: {
    total: number;
    available: number;
  };

  // Oxygen Status
  oxygen: {
    available: boolean;
    cylindersRemaining?: number; // Optional
    estimatedHours?: number;     // Optional
  };

  // Disease Admissions (aggregated counts)
  diseaseCounts: {
    malaria: number;
    dengue: number;
    covidLike: number;   // ILI/SARI
    other: Record<string, number>; // Flexible for outbreaks
  };

  // Status Flags
  status: 'draft' | 'submitted';
  isEdited: boolean;
  previousVersionId?: string;
}

// Hospital User
interface HospitalUser {
  id: string;
  email: string;
  hospitalId: string;
  hospitalName: string;
  role: 'admin' | 'staff';
  lastLogin?: string;
}
```

6. Pages / Screens Design

Screen Overview

Screen	Route	Purpose
Login	`/login`	Hospital authentication
Dashboard	`/dashboard`	Overview, quick actions, recent submissions
New Report	`/report/new`	Daily capacity form
Edit Report	`/report/[id]/edit`	Edit previous submission
History	`/history`	List of all past reports
Profile	`/profile`	Hospital details, settings

Screen Flow Diagram

7. Detailed Screen Specifications

Screen 1: Login (`/login`)

Purpose: Authenticate hospital users

UI Elements:

- Hospital email input
- Password input
- "Remember me" checkbox
- Login button
- Demo credentials hint (for prototype)

Demo Credentials:

```
Email: civil@hospital.gov
Password: demo123

Email: district@hospital.gov
Password: demo123
```

Logic:

- POST to `/api/auth/login`
- Receive JWT token
- Store in HTTP-only cookie or localStorage
- Redirect to dashboard

API Response:

```
{
  "token": "jwt_token_here",
  "user": {
    "hospitalId": "H001",
    "hospitalName": "Civil Hospital Solapur",
    "email": "civil@hospital.gov"
  }
}
```

Screen 2: Dashboard (`/dashboard`)

Purpose: Main hub for hospital staff

UI Elements:

Header Section:

- Welcome message with hospital name
- Current date
- Last submission status

Stats Cards:

- 📊 **Last Report**: Date of most recent submission
- 🛏️ **Beds Available**: From last report
- 🏥 **ICU Available**: From last report
- 🌬️ **Ventilators**: From last report
- 💧 **Oxygen**: Status indicator

Quick Actions:

- ➕ **New Daily Report** (primary CTA)

- 📋 **View History**
- ⚙️ **Profile Settings**

Recent Submissions Table:

Date	Beds Avail.	ICU Avail.	O2 Status	Actions
2026-03-10	25/100	2/10	✅	View/Edit
2026-03-09	18/100	1/10	✅	View

Design Principles:

- Clean, professional interface
- Color-coded status (green/yellow/red)
- Mobile-responsive layout

Screen 3: New Report Form (`/report/new`)

Purpose: Submit daily capacity and disease data

Form Design:

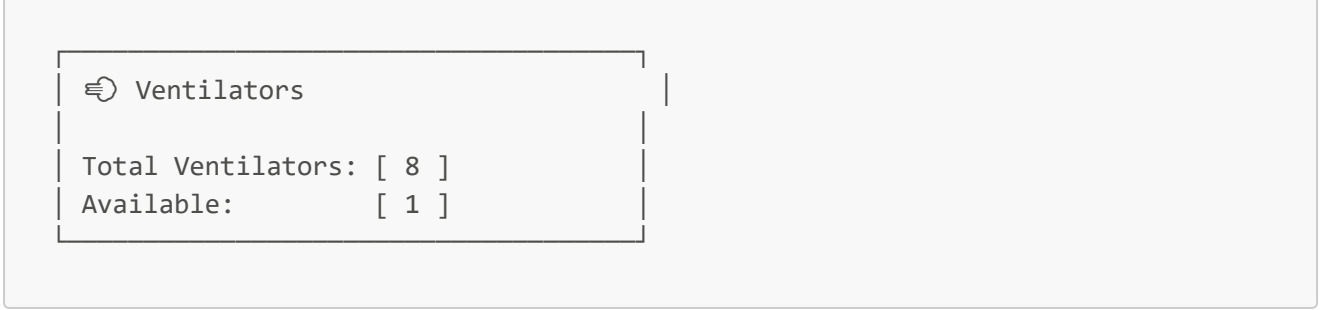
Section 1: Bed Capacity

```
┌─────────────────────────────────────┐
│ 🛏️ Bed Capacity                     │
│                                     │
│ Total Beds:    [ 100 ]              │
│ Available Beds:[ 25  ]              │
│                                     │
│ 💡 Occupied: 75 (calculated)        │
└─────────────────────────────────────┘
```

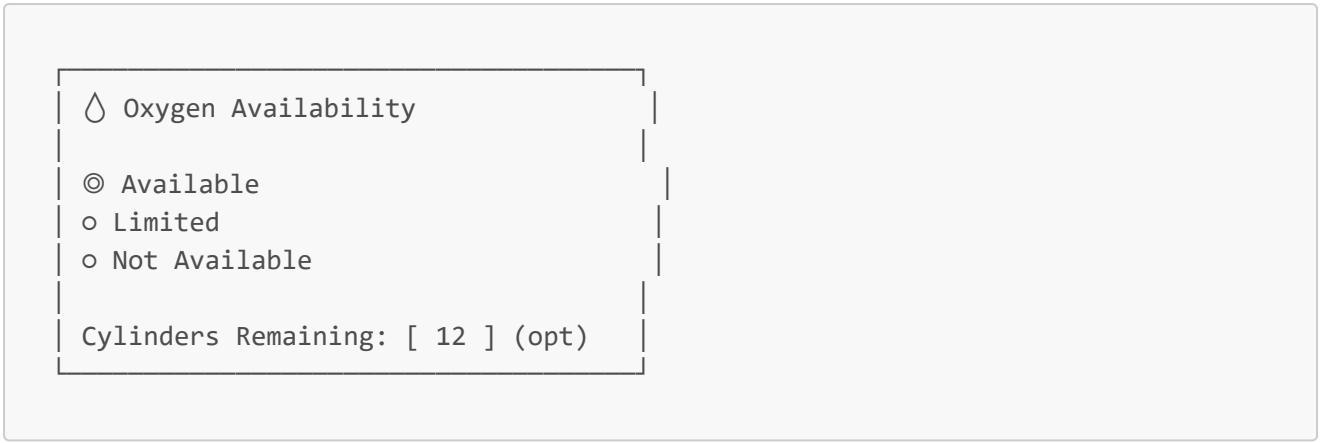
Section 2: ICU Capacity

```
┌─────────────────────────────────────┐
│ 🏥 ICU Capacity                     │
│                                     │
│ Total ICU Beds: [ 10 ]              │
│ Available ICU:  [ 2  ]              │
└─────────────────────────────────────┘
```

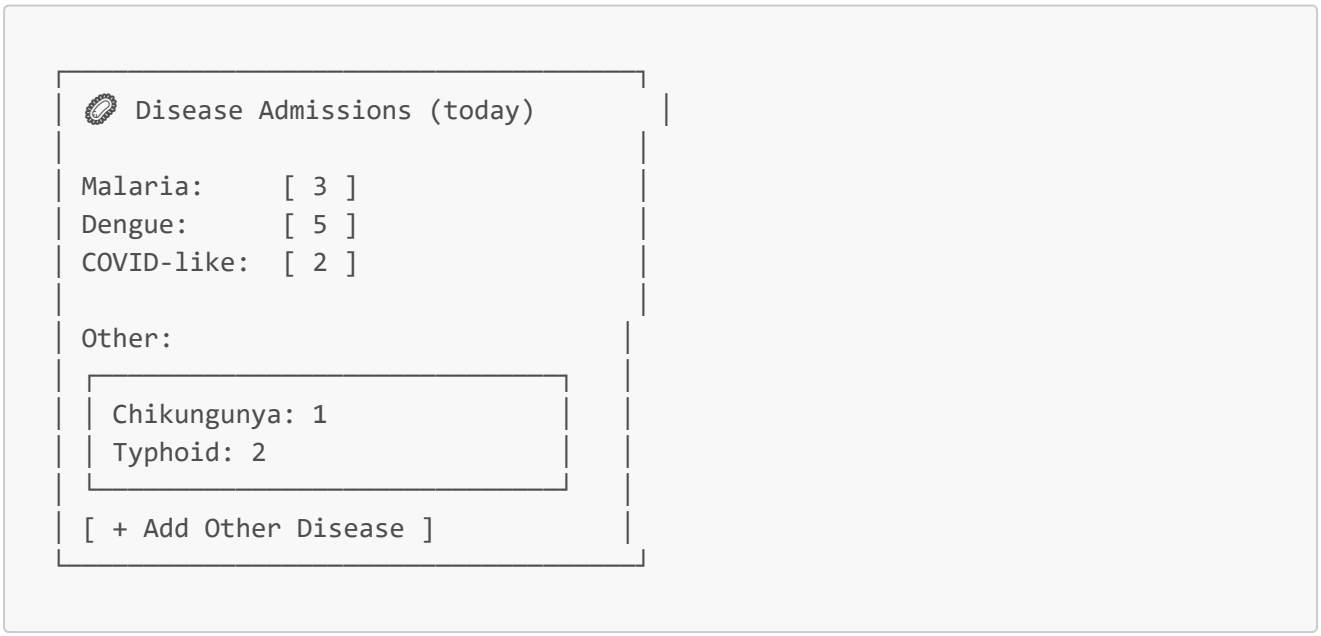
Section 3: Ventilators



Section 4: Oxygen Status



Section 5: Disease Admissions



Form Actions:

- [Save Draft] - Store locally (optional)
- [Submit Report] - POST to API
- [Cancel] - Return to dashboard

Key Features:

- **Pre-fill from yesterday** – Auto-populate with last submission
- **Validation** – Available ≤ Total for each category

- **Calculated fields** – Occupied shown automatically
- **Mobile-friendly** – Stacked on small screens

Screen 4: History (`/history`)

Purpose: View all past submissions

UI Elements:

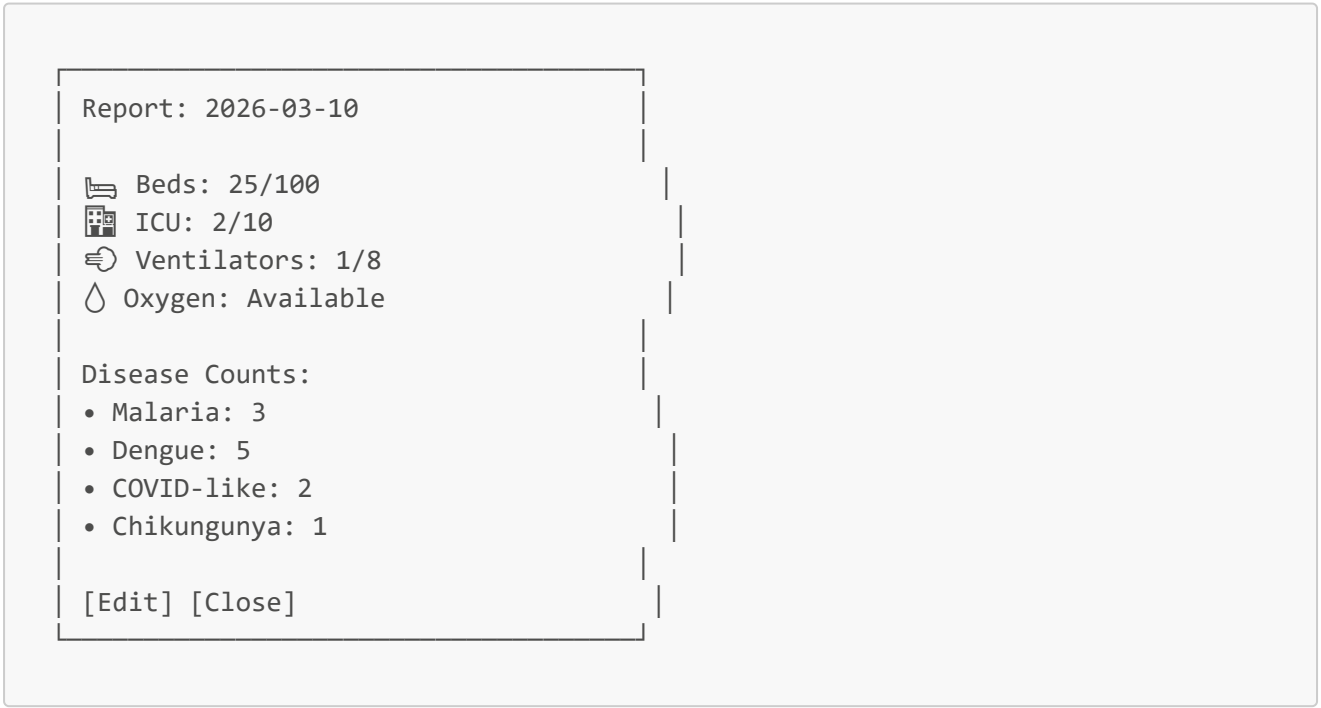
Filters:

- Date range picker
- Status filter (All/Submitted/Draft)

Table Columns:

Date	Submitted By	Beds (Avail/Total)	ICU (Avail/Total)	O2	Actions
2026-03-10	admin@...	25/100	2/10	☑	View
2026-03-09	admin@...	18/100	1/10	☑	View
2026-03-08	admin@...	42/100	5/10	☑	View

Action Modal (View Report):



Edit Restriction: Only allow editing of **today's report** or most recent day

---

Screen 5: Profile (`/profile`)

Purpose: View/update hospital details

UI Elements:

- Hospital name (read-only)
- Contact email
- Phone number
- Change password
- API token (for future integration)

---

8. API Integration

Backend Endpoints

Endpoint	Method	Purpose	Request/Response
`/api/hospitals/login`	POST	Authentication	`{ email, password }` → `{ token, user }`
`/api/hospitals/report`	POST	Submit report	`CapacityReport` → `{ success, reportId }`
`/api/hospitals/reports`	GET	List hospital reports	Query params: `fromDate`, `toDate` → `CapacityReport[]`
`/api/hospitals/report/:id`	GET	Get single report	→ `CapacityReport`
`/api/hospitals/report/:id`	PUT	Update report	`CapacityReport` → `{ success }`
`/api/hospitals/latest`	GET	Get most recent report	→ `CapacityReport`

Frontend Data Flow

Using TanStack Query for efficient data management:

```
// Query hooks
const useLatestReport = (hospitalId) => {
  return useQuery({
    queryKey: ['report', 'latest', hospitalId],
    queryFn: () => fetchLatestReport(hospitalId),
    staleTime: 5 * 60 * 1000, // 5 minutes
  });
};

const useSubmitReport = () => {
  return useMutation({
    mutationFn: (report) => submitReport(report),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['reports'] });
      queryClient.invalidateQueries({ queryKey: ['report', 'latest'] });
```

```
    }
  });
};
```

Form Pre-fill Strategy

```
// On new report page
const { data: latestReport } = useLatestReport(hospitalId);

const defaultValues = latestReport ? {
  beds: {
    total: latestReport.beds.total,
    available: latestReport.beds.available // Keep same available?
  },
  // ... other fields
} : initialEmptyState;
```

---

9. Implementation Priority (13 Days)

Priority	Screens/Features	Day Allocation
P1 (Must Have)	Project setup, Login, Dashboard, New Report form, API integration	Days 1-4
P2 (Should Have)	Pre-fill from yesterday, Form validation, Submission feedback	Days 5-6
P3 (Nice to Have)	History view, Edit functionality, Profile page	Days 7-8
Buffer	Testing, bug fixes, polish	Days 9-13

Day-by-Day Breakdown

Day	Focus
1	Next.js project setup, Tailwind, shadcn/ui, folder structure
2	Login page, authentication flow, protected routes
3	Dashboard layout, stats cards, navigation
4	New Report form – UI components
5	Form validation, pre-fill logic
6	API integration (POST reports)

Day	Focus
7	History list view
8	Edit functionality, single report view
9	Profile page, logout
10	Error handling, loading states
11	Responsive design testing
12	Bug fixes, polish
13	Final testing, video recording

10. Project Structure

```
hospital-portal/
├── public/
├── src/
│   ├── app/
│   │   ├── layout.tsx
│   │   ├── page.tsx (redirect to dashboard)
│   │   ├── login/
│   │   │   └── page.tsx
│   │   ├── dashboard/
│   │   │   └── page.tsx
│   │   ├── report/
│   │   │   ├── new/
│   │   │   │   └── page.tsx
│   │   │   └── [id]/
│   │   │       ├── page.tsx (view)
│   │   │       └── edit/
│   │   │           └── page.tsx
│   │   ├── history/
│   │   │   └── page.tsx
│   │   └── profile/
│   │       └── page.tsx
│   ├── components/
│   │   ├── layout/
│   │   │   ├── Header.tsx
│   │   │   └── Sidebar.tsx
│   │   ├── forms/
│   │   │   ├── CapacityForm.tsx
│   │   │   ├── BedSection.tsx
│   │   │   └── DiseaseCounts.tsx
│   │   ├── dashboard/
│   │   │   ├── StatsCards.tsx
│   │   │   └── RecentSubmissions.tsx
│   │   └── ui/ (shadcn components)
│   ├── hooks/
```

```
│   │   ├── useAuth.ts
│   │   ├── useReports.ts
│   │   └── useSubmitReport.ts
│   ├── lib/
│   │   ├── api/
│   │   │   ├── client.ts
│   │   │   └── endpoints.ts
│   │   ├── auth/
│   │   │   └── auth.ts
│   │   ├── validations/
│   │   │   └── reportSchema.ts (Zod)
│   │   └── utils/
│   │       ├── formatting.ts
│   │       └── constants.ts
│   ├── types/
│   │   └── index.ts
│   └── middleware.ts (auth)
├── .env.local
├── tailwind.config.js
└── package.json
```

---

11. Integration with Arogya-SMC Ecosystem

Data Flow

```
[Hospital Portal] → [API Gateway] → [FHIR Standardization] → [Central Database]
                         ↓                    ↓                    ↓
                   [JWT Auth]          [SNOMED Mapping]     [Ward Indexing]
```

How Hospital Data Powers Other Modules

Module	Data Used	Impact
Citizen App	Bed/ICU availability	Citizens see real-time availability
Admin Dashboard	All capacity + disease counts	Resource monitoring, outbreak detection
Analytics Engine	Historical capacity + disease	Trend analysis, forecasting

FHIR Alignment

Hospital reports should be transformed to FHIR Observation resources:

```
{
  "resourceType": "Observation",
  "code": {
    "coding": [{
```

```
        "system": "http://loinc.org",
        "code": "69524-4",
        "display": "Hospital bed census"
      }]
    },
    "valueQuantity": {
      "value": 25,
      "unit": "beds"
    },
    "effectiveDateTime": "2026-03-10",
    "extension": [{
      "url": "http://example.org/fhir/StructureDefinition/bed-type",
      "valueCodeableConcept": {
        "coding": [{
          "system": "http://terminology.hl7.org/CodeSystem/bed-type",
          "code": "ICU"
        }]
      }
    }]
  }
```

---

12. Success Criteria for Demo Video

The 4-5 minute demo should clearly show:

1. **Login** – Hospital admin logs in
2. **Dashboard** – Overview with last submission stats
3. **New Report** – Fill form with:
    - Bed capacity
    - ICU availability
    - Ventilator counts
    - Oxygen status
    - Disease admissions
4. **Pre-fill** – Show that yesterday's values auto-populate
5. **Validation** – Error shown if available > total
6. **Submit** – Success message appears
7. **History** – New report appears in list
8. **Citizen Impact** – (Optional) Show citizen app reflecting updated availability

---

13. Key Design Principles (Learning from Existing Systems)

What to Emulate

✅ **Clean, professional interface** – Hospital staff are busy; minimize cognitive load
✅ **Mobile-responsive** – Admins may access from tablets/phones
✅ **Form validation** – Prevent data entry errors

✅ **Audit trail** – Track who submitted what when
✅ **Pre-fill** – Reduce repetitive typing

What to Avoid

❌ **Complex multi-step wizards** – One scrollable form works better
❌ **Slow loading times** – Optimize bundle size
❌ **Unclear error messages** – Be specific about what's wrong
❌ **Missing confirmation** – Always show submission success

---

14. Conclusion

The Arogya-SMC Hospital Portal provides a simple yet powerful interface for Solapur's hospitals to report critical capacity data. By following industry standards for bed reporting and incorporating lessons from existing healthcare dashboards , this portal ensures:

- **Hospitals** can report quickly with minimal effort
- **Citizens** get accurate, real-time availability information
- **SMC officials** gain visibility into city-wide healthcare resources
- **The system** remains scalable and standards-aligned